

700106 / 700120 Lab Book

Vulkan Exercises

Exercise 1: IMPLEMENTING AND CONTROLLING THE DEPTH BUFFER

Solution: First, I added helper functions `findSupportedFormat` and `findDepthFormat` to locate a depth format compatible with the GPU (e.g., `VK_FORMAT_D32_SFLOAT`). I then implemented `createDepthResources` to create the `VkImage`, `VkDeviceMemory`, and `VkImageView` specifically for the depth attachment .

In the pipeline creation, I set up `VkPipelineDepthStencilStateCreateInfo` with `depthTestEnable` and `depthWriteEnable` set to `VK_TRUE`.

Crucially, to fulfill the lab's requirement of observing these effects, I enabled Dynamic State for the depth settings. As seen in my code, I added `VK_DYNAMIC_STATE_DEPTH_TEST_ENABLE` and `VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE` to the dynamic states vector. This allowed me to add keyboard input logic (keys T/G and Y/H) to toggle these states in real-time using `vkCmdSetDepthTestEnable` and `vkCmdSetDepthWriteEnable` inside the command buffer recording loop

Reflection: This exercise demonstrated the vital role of the Z-buffer. Before this, triangles drew in the order they were defined; now, they draw based on distance. Implementing the dynamic state was particularly interesting because it showed that not all pipeline states are immutable; Vulkan allows specific states to be changed without recreating the entire pipeline, which is efficient for debugging or specific rendering effects.

Question: I used `VK_FORMAT_D32_SFLOAT` for my depth buffer. Is there a significant performance or memory bandwidth advantage to using a lower precision format like `D16_UNORM` for simple scenes, or do modern GPUs handle 32-bit depth values so efficiently that the difference is negligible?

```
VkPhysicalDeviceExtendedDynamicStateFeaturesEXT extDyn{};
extDyn.sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_EXTENDED_DYNAMIC_STATE_FEATURES_EXT;
extDyn.extendedDynamicState = VK_TRUE;

extDyn.pNext = &sync2Features;
dynamicRenderingFeatures.pNext = &extDyn;
```

```

std::vector<VkDynamicState> dynamicStates = {
    VK_DYNAMIC_STATE_VIEWPORT,
    VK_DYNAMIC_STATE_SCISSOR,
    VK_DYNAMIC_STATE_DEPTH_TEST_ENABLE,
    VK_DYNAMIC_STATE_DEPTH_WRITE_ENABLE
};

VkPipelineDynamicStateCreateInfo dynamicState{};
dynamicState.sType = VK_STRUCTURE_TYPE_PIPELINE_DYNAMIC_STATE_CREATE_INFO;
dynamicState.dynamicStateCount = static_cast<uint32_t>(dynamicStates.size());
dynamicState.pDynamicStates = dynamicStates.data();

```

```

517
518 // Depth test enable/disable (T = on, G = off)
519 if (glfwGetKey(window, GLFW_KEY_T) == GLFW_PRESS) depthTestEnabled = true;
520 if (glfwGetKey(window, GLFW_KEY_G) == GLFW_PRESS) depthTestEnabled = false;
521
522 // Depth write enable/disable (Y = on, H = off)
523 if (glfwGetKey(window, GLFW_KEY_Y) == GLFW_PRESS) depthWriteEnabled = true;
524 if (glfwGetKey(window, GLFW_KEY_H) == GLFW_PRESS) depthWriteEnabled = false;
525 }
526

```

```

10 vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, graphicsPipeline);
11
12 vkCmdSetDepthTestEnable(commandBuffer, depthTestEnabled ? VK_TRUE : VK_FALSE);
13 vkCmdSetDepthWriteEnable(commandBuffer, depthWriteEnabled ? VK_TRUE : VK_FALSE);
14
15 // NEW: Set dynamic viewport and scissor (required since they are dynamic states)
16

```

```

8
9 // --- ADD THIS NEW FUNCTION ---
10 VkFormat findSupportedFormat(const std::vector<VkFormat>& candidates, VkImageTiling tiling, VkFormatFeatureFlags features) {
11     for (VkFormat format : candidates) {
12         VkFormatProperties props;
13         vkGetPhysicalDeviceFormatProperties(physicalDevice, format, &props);
14
15         if (tiling == VK_IMAGE_TILING_LINEAR && (props.linearTilingFeatures & features) == features) {
16             return format;
17         }
18         else if (tiling == VK_IMAGE_TILING_OPTIMAL && (props.optimalTilingFeatures & features) == features) {
19             return format;
20         }
21     }
22     throw std::runtime_error("failed to find supported format!");
23 }
24
25 // --- AND THIS ONE ---
26 VkFormat findDepthFormat() {
27     return findSupportedFormat(
28         { VK_FORMAT_D32_SFLOAT, VK_FORMAT_D32_SFLOAT_S8_UINT, VK_FORMAT_D24_UNORM_S8_UINT },
29         VK_IMAGE_TILING_OPTIMAL,
30         VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
31     );
32 }
33

```

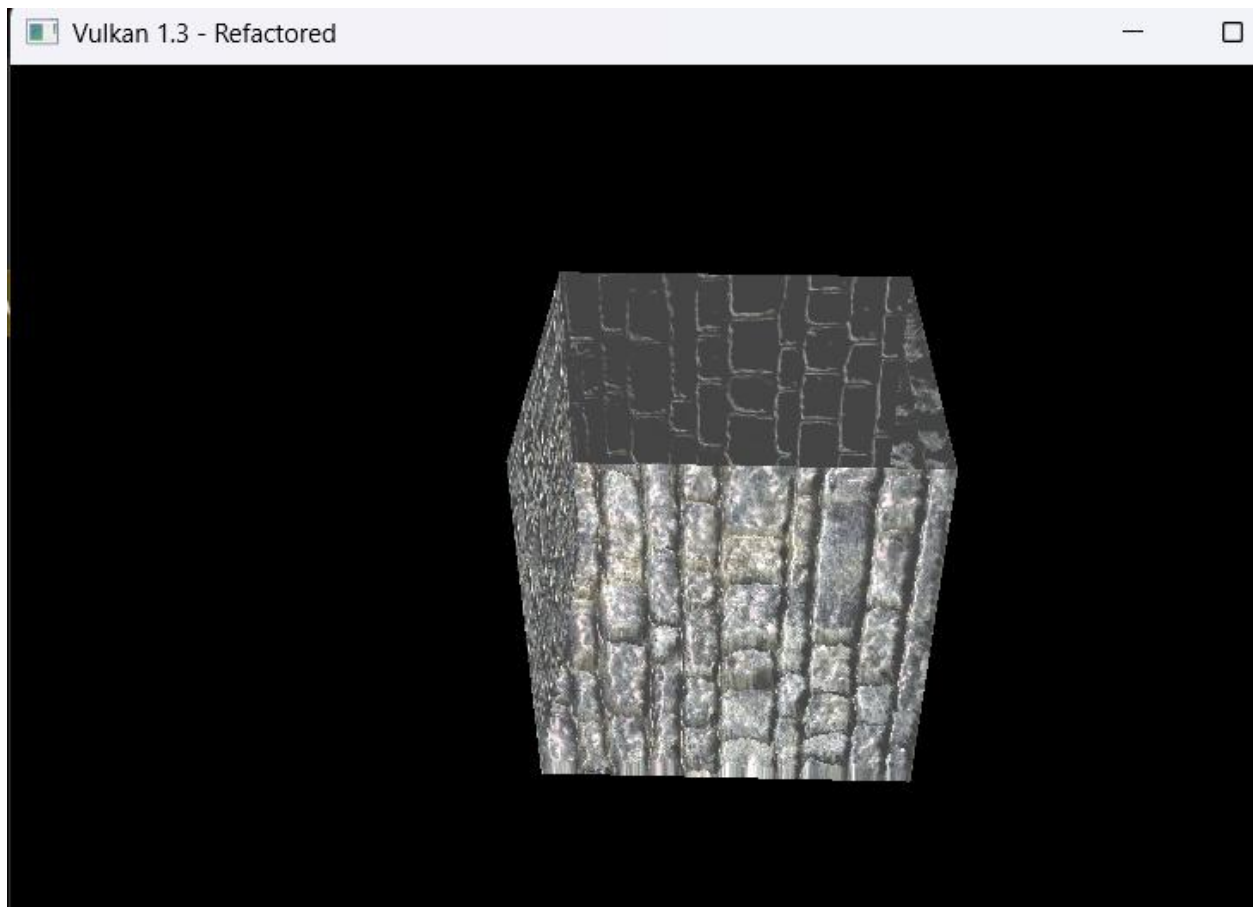
```

842
843 void HelloTriangleApplication::createDepthResources()
844 {
845     VkExtent2D extent = swapChainExtent;
846
847     // Create the Image
848     VkImageCreateInfo imageInfo{};
849     imageInfo.sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO;
850     imageInfo.imageType = VK_IMAGE_TYPE_2D;
851     imageInfo.extent.width = extent.width;
852     imageInfo.extent.height = extent.height;
853     imageInfo.extent.depth = 1;
854     imageInfo.mipLevels = 1;
855     imageInfo.arrayLayers = 1;
856     imageInfo.format = depthFormat;
857     imageInfo.tiling = VK_IMAGE_TILING_OPTIMAL;
858     imageInfo.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
859     imageInfo.usage = VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT;
860     imageInfo.samples = VK_SAMPLE_COUNT_1_BIT;
861     imageInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
862
863     if (vkCreateImage(device, &imageInfo, nullptr, &depthImage) != VK_SUCCESS) {
864         throw std::runtime_error("failed to create depth image!");
865     }
866
867     // Allocate Memory
868     VkMemoryRequirements memRequirements;
869     vkGetImageMemoryRequirements(device, depthImage, &memRequirements);
870
871     VkMemoryAllocateInfo allocInfo{};
872     allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
873     allocInfo.allocationSize = memRequirements.size;
874     allocInfo.memoryTypeIndex = findMemoryType(memRequirements.memoryTypeBits, VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT);
875
876     if (vkAllocateMemory(device, &allocInfo, nullptr, &depthImageMemory) != VK_SUCCESS) {
877         throw std::runtime_error("failed to allocate depth image memory!");
878     }
879
880     // Bind Memory
881     vkBindImageMemory(device, depthImage, depthImageMemory, 0);
882
883     // Create the Image View
884     depthImageView = createImageView(depthImage, depthFormat, VK_IMAGE_ASPECT_DEPTH_BIT);
885
886     // Transition Layout
887     transitionImageLayout(depthImage,

```

 Vulkan 1.3 - Refactored





Exercise 2: IMPLEMENTING THE SKYBOX

Solution: I created a `createSkyboxCubemap` function that loads 6 separate image files (right, left, top, bottom, front, back) using `stbi_load`. I copied these into a single `VkImage` created with the `VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT` flag and `arrayLayers = 6`. I handled the buffer copying carefully, calculating offsets to place each face into the correct array layer.

For the rendering pipeline (`createSkyboxPipeline`), I set `cullMode` to `VK_CULL_MODE_FRONT_BIT`. This is because we are viewing the cube from the *inside*, so the winding order is reversed relative to the camera. I also set the depth comparison operator to `VK_COMPARE_OP_LESS_OR_EQUAL` to ensure the skybox renders behind everything else.

Finally, I updated the descriptor sets to bind this cubemap to a `samplerCube` in the shader, allowing the fragment shader to sample the texture using a 3D direction vector rather than 2D UVs.

Reflection: This task highlighted the difference between standard 2D textures and Cubemaps. The most tricky part was ensuring the memory layout and staging buffer copies were correct for the 6 array layers. Visually, the result is very effective; the scene no longer floats in a black void but feels grounded in a world.

Question: In the vertex shader, we often modify the view matrix (removing the translation column) to make the skybox appear infinitely far away. My code draws a physical cube. Is there a performance benefit to using a "full-screen triangle" approach for the skybox and reconstructing the ray direction, versus drawing actual cube geometry as I did here?

```
363 //Skyinfo
364 VkDescriptorImageInfo skyInfo{};
365 skyInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
366 skyInfo.imageView = skyboxImageView;
367 skyInfo.sampler = skyboxSampler;
368
369 //Skybox
370 std::array<VkWriteDescriptorSet, 5> writes{};
371 // (writes[0..3] unchanged)
372 writes[4].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
373 writes[4].dstSet = descriptorSets[i];
374 writes[4].dstBinding = 4;
375 writes[4].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
376 writes[4].descriptorCount = 1;
377 writes[4].pImageInfo = &skyInfo;
```

```

548
549 void HelloTriangleApplication::createSkyboxCubemap()
550 {
551     int w[6], h[6], c[6];
552     std::vector<stbi_uc*> facePixels(6);
553     for (int i = 0; i < 6; ++i) {
554         facePixels[i] = stbi_load(skyboxPaths[i].c_str(), &w[i], &h[i], &c[i], STBI_rgb_alpha);
555         if (!facePixels[i]) throw std::runtime_error("Failed to load cubemap face: " + skyboxPaths[i]);
556         if (w[i] != w[0] || h[i] != h[0]) throw std::runtime_error("Cubemap faces must have same dimensions");
557     }
558     uint32_t faceWidth = (uint32_t)w[0];
559     uint32_t faceHeight = (uint32_t)h[0];
560     VkDeviceSize faceSize = VkDeviceSize(faceWidth) * faceHeight * 4;
561
562     // Create cube image (arrayLayers = 6)
563     VkImageCreateInfo img{};
564     img.sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO;
565     img.flags = VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT;
566     img.imageType = VK_IMAGE_TYPE_2D;
567     img.format = VK_FORMAT_R8G8B8A8_SRGB;
568     img.extent = { faceWidth, faceHeight, 1 };
569     img.mipLevels = 1;
570     img.arrayLayers = 6;
571     img.samples = VK_SAMPLE_COUNT_1_BIT;
572     img.tiling = VK_IMAGE_TILING_OPTIMAL;
573     img.usage = VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT;
574     img.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
575     img.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
576     if (vkCreateImage(device, &img, nullptr, &skyboxImage) != VK_SUCCESS)
577         throw std::runtime_error("Failed to create cubemap image");
578
579     VkMemoryRequirements req;
580     vkGetImageMemoryRequirements(device, skyboxImage, &req);
581     VkMemoryAllocateInfo alloc{};
582     alloc.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
583     alloc.allocationSize = req.size;
584     alloc.memoryTypeIndex = findMemoryType(req.memoryTypeBits, VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT);
585     if (vkAllocateMemory(device, &alloc, nullptr, &skyboxImageMemory) != VK_SUCCESS)
586         throw std::runtime_error("Failed to allocate cubemap memory");
587     vkBindImageMemory(device, skyboxImage, skyboxImageMemory, 0);

```

```

589 // Staging upload per face
590 for (int i = 0; i < 6; ++i) {
591     VkBuffer staging;
592     VkDeviceMemory stagingMem;
593     createBuffer(faceSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
594                 VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
595                 staging, stagingMem);
596     void* data;
597     vkMapMemory(device, stagingMem, 0, faceSize, 0, &data);
598     memcpy(data, facePixels[i], (size_t)faceSize);
599     vkUnmapMemory(device, stagingMem);
600     stbi_image_free(facePixels[i]);
601
602     // Transition (only once before first copy)
603     if (i == 0) {
604         transitionImageLayout(skyboxImage,
605                             VK_IMAGE_LAYOUT_UNDEFINED,
606                             VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
607                             VK_IMAGE_ASPECT_COLOR_BIT);
608     }
609
610     VkCommandBuffer cmd = beginSingleTimeCommands();
611     VkBufferImageCopy region{};
612     region.imageSubresource.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
613     region.imageSubresource.mipLevel = 0;
614     region.imageSubresource.baseArrayLayer = i;
615     region.imageSubresource.layerCount = 1;
616     region.imageExtent = { faceWidth, faceHeight, 1 };
617     vkCmdCopyBufferToImage(cmd, staging, skyboxImage,
618                          VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, 1, &region);
619     endSingleTimeCommands(cmd);
620
621     vkDestroyBuffer(device, staging, nullptr);
622     vkFreeMemory(device, stagingMem, nullptr);
623 }
624
625 transitionImageLayout(skyboxImage,
626                     VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
627                     VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL,
628                     VK_IMAGE_ASPECT_COLOR_BIT);
629
630 VkImageViewCreateInfo viewInfo{};
631 viewInfo.sType = VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO;
632 viewInfo.image = skyboxImage;
633 viewInfo.viewType = VK_IMAGE_VIEW_TYPE_CUBE;
634 viewInfo.format = VK_FORMAT_R8G8B8A8_SRGB;
635 viewInfo.subresourceRange.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;

```

```

630 VkImageViewCreateInfo viewInfo{};
631 viewInfo.sType = VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO;
632 viewInfo.image = skyboxImage;
633 viewInfo.viewType = VK_IMAGE_VIEW_TYPE_CUBE;
634 viewInfo.format = VK_FORMAT_R8G8B8A8_SRGB;
635 viewInfo.subresourceRange.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
636 viewInfo.subresourceRange.baseMipLevel = 0;
637 viewInfo.subresourceRange.levelCount = 1;
638 viewInfo.subresourceRange.baseArrayLayer = 0;
639 viewInfo.subresourceRange.layerCount = 6;
640 if (vkCreateImageView(device, &viewInfo, nullptr, &skyboxImageView) != VK_SUCCESS)
641     throw std::runtime_error("Failed to create cubemap image view");
642
643 VkSamplerCreateInfo samp{};
644 samp.sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO;
645 samp.magFilter = VK_FILTER_LINEAR;
646 samp.minFilter = VK_FILTER_LINEAR;
647 samp.addressModeU = VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE;
648 samp.addressModeV = VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE;
649 samp.addressModeW = VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE;
650 samp.mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR;
651 samp.maxLod = 0.0f;
652 if (vkCreateSampler(device, &samp, nullptr, &skyboxSampler) != VK_SUCCESS)
653     throw std::runtime_error("Failed to create cubemap sampler");
654 }

```



```

2396
2397     VkDescriptorImageInfo skyInfo{};
2398     skyInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
2399     skyInfo.imageView = skyboxImageView;
2400     skyInfo.sampler = skyboxSampler;
2401
2402     std::array<VkWriteDescriptorSet, 4> writes{};
2403     // existing 3 writes plus:
2404     writes[3].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
2405     writes[3].dstSet = descriptorSets[i];
2406     writes[3].dstBinding = 4;
2407     writes[3].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
2408     writes[3].descriptorCount = 1;
2409     writes[3].pImageInfo = &skyInfo;
2410

```

```

// --- Skybox pass ---
vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, skyboxPipeline);
// dynamic viewport/scissor already set later; set them now for clarity
VkViewport vp{ 0.0f, 0.0f, (float)swapChainExtent.width, (float)swapChainExtent.height, 0.0f, 1.0f };
VkRect2D sc{ {0,0}, swapChainExtent };
vkCmdSetViewport(commandBuffer, 0, 1, &vp);
vkCmdSetScissor(commandBuffer, 0, 1, &sc);
VkDeviceSize SkyOffsets[] = { 0 };
vkCmdBindVertexBuffers(commandBuffer, 0, 1, &vertexBuffer, SkyOffsets);
vkCmdBindIndexBuffer(commandBuffer, indexBuffer, 0, VK_INDEX_TYPE_UINT16);
vkCmdBindDescriptorSets(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS,
    pipelineLayout, 0, 1, &descriptorSets[currentFrame], 0, nullptr);
// No push constants needed for skybox (model unused or identity)
vkCmdDrawIndexed(commandBuffer, static_cast<uint32_t>(indices.size()), 1, 0, 0, 0);

```



```

658
659 void HelloTriangleApplication::createSkyboxPipeline()
660 {
661     auto vertCode = readFile("shaders/skybox.vert.spv");
662     auto fragCode = readFile("shaders/skybox.frag.spv");
663     VkShaderModule vert = createShaderModule(vertCode);
664     VkShaderModule frag = createShaderModule(fragCode);
665
666     VkPipelineShaderStageCreateInfo stages[2]{};
667     stages[0].sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
668     stages[0].stage = VK_SHADER_STAGE_VERTEX_BIT;
669     stages[0].module = vert;
670     stages[0].pName = "main";
671     stages[1].sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
672     stages[1].stage = VK_SHADER_STAGE_FRAGMENT_BIT;
673     stages[1].module = frag;
674     stages[1].pName = "main";
675
676     auto bindingDescription = Vertex::getBindingDescription();
677     auto attributeDescriptions = Vertex::getAttributeDescriptions(); // we only use location 0
678
679     VkPipelineVertexInputStateCreateInfo vi{};
680     vi.sType = VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO;
681     vi.vertexBindingDescriptionCount = 1;
682     vi.pVertexBindingDescriptions = &bindingDescription;
683     vi.vertexAttributeDescriptionCount = 1; // only position
684     vi.pVertexAttributeDescriptions = &attributeDescriptions[0];
685
686     VkPipelineInputAssemblyStateCreateInfo ia{};
687     ia.sType = VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_INFO;
688     ia.topology = VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST;
689
690     VkPipelineViewportStateCreateInfo vp{};
691     vp.sType = VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO;
692     vp.viewportCount = 1;
693     vp.scissorCount = 1;
694
695     VkPipelineRasterizationStateCreateInfo rs{};
696     rs.sType = VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO;
697     rs.polygonMode = VK_POLYGON_MODE_FILL;
698     rs.cullMode = VK_CULL_MODE_FRONT_BIT; // inside the cube
699     rs.frontFace = VK_FRONT_FACE_COUNTER_CLOCKWISE;
700     rs.lineWidth = 1.0f;

```

```

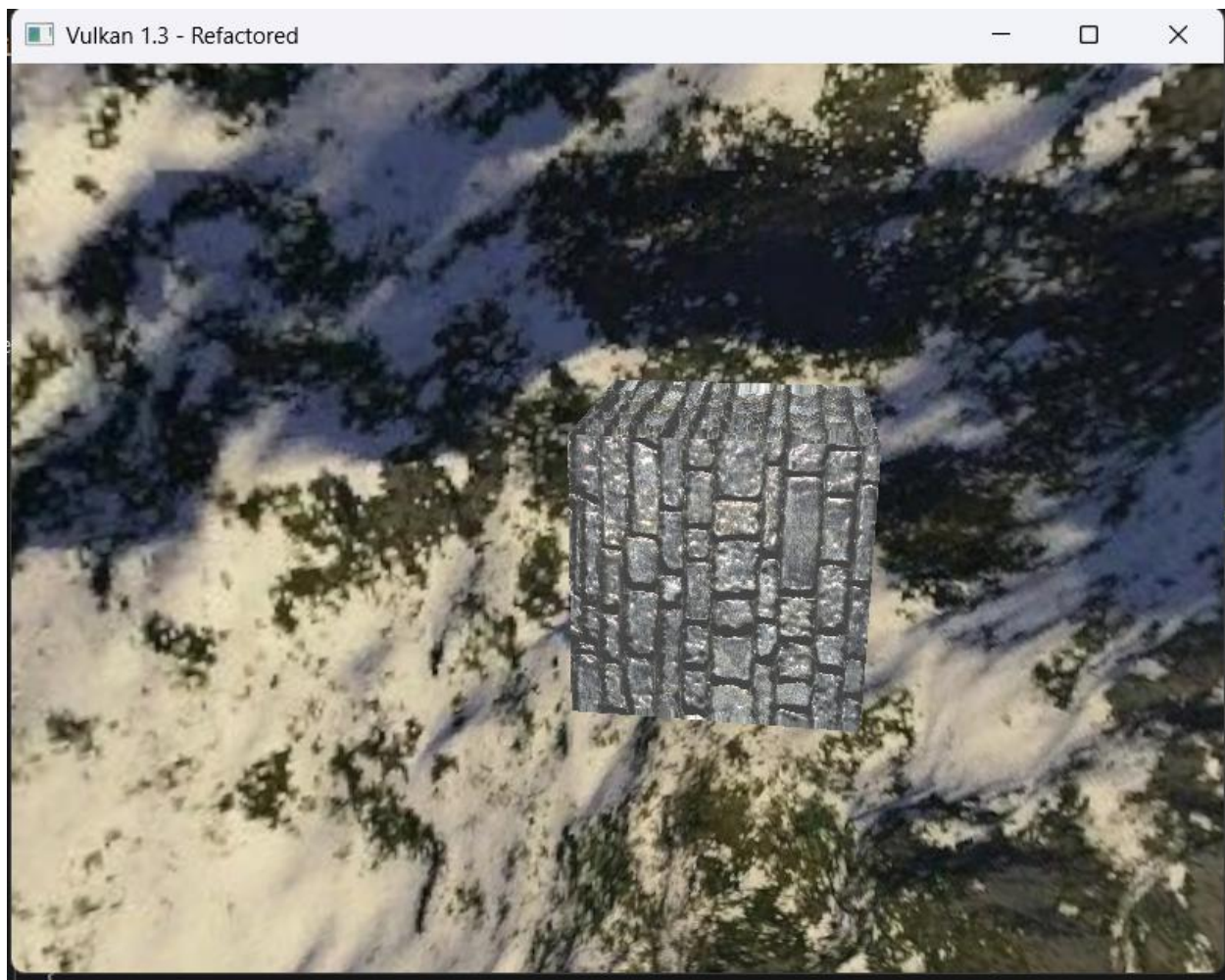
701
702     VkPipelineMultisampleStateCreateInfo ms{};
703     ms.sType = VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO;
704     ms.rasterizationSamples = VK_SAMPLE_COUNT_1_BIT;
705
706     VkPipelineDepthStencilStateCreateInfo ds{};
707     ds.sType = VK_STRUCTURE_TYPE_PIPELINE_DEPTH_STENCIL_STATE_CREATE_INFO;
708     ds.depthTestEnable = VK_TRUE;
709     ds.depthWriteEnable = VK_FALSE;
710     ds.depthCompareOp = VK_COMPARE_OP_LESS_OR_EQUAL;
711     ds.stencilTestEnable = VK_FALSE;
712
713     VkPipelineColorBlendAttachmentState cbAtt{};
714     cbAtt.colorWriteMask = VK_COLOR_COMPONENT_R_BIT | VK_COLOR_COMPONENT_G_BIT |
715         VK_COLOR_COMPONENT_B_BIT | VK_COLOR_COMPONENT_A_BIT;
716
717     VkPipelineColorBlendStateCreateInfo cb{};
718     cb.sType = VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_STATE_CREATE_INFO;
719     cb.attachmentCount = 1;
720     cb.pAttachments = &cbAtt;
721
722     std::vector<VkDynamicState> dyn = {
723         VK_DYNAMIC_STATE_VIEWPORT,
724         VK_DYNAMIC_STATE_SCISSOR
725     };
726     VkPipelineDynamicStateCreateInfo dynInfo{};
727     dynInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_DYNAMIC_STATE_CREATE_INFO;
728     dynInfo.dynamicStateCount = (uint32_t)dyn.size();
729     dynInfo.pDynamicStates = dyn.data();
730
731     // Reuse existing pipelineLayout (contains needed descriptor set & push constants not used here)
732     VkPipelineRenderingCreateInfo renderInfo{};
733     renderInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_RENDERING_CREATE_INFO;
734     renderInfo.colorAttachmentCount = 1;
735     renderInfo.pColorAttachmentFormats = &swapChainImageFormat;
736     renderInfo.depthAttachmentFormat = depthFormat;
737
738     VkGraphicsPipelineCreateInfo gp{};
739     gp.sType = VK_STRUCTURE_TYPE_GRAPHICS_PIPELINE_CREATE_INFO;
740     gp.pNext = &renderInfo;
741     gp.stageCount = 2;
742     gp.pStages = stages;
743     gp.pVertexInputState = &vi;
744     gp.pInputAssemblyState = &ia;
745     gp.pViewportState = &vp;
746     gp.pRasterizationState = &rs;
747     gp.pMultisampleState = &ms;
748     gp.pDepthStencilState = &ds;

```

```

743     gp.pVertexInputState = &vi;
744     gp.pInputAssemblyState = &ia;
745     gp.pViewportState = &vp;
746     gp.pRasterizationState = &rs;
747     gp.pMultisampleState = &ms;
748     gp.pDepthStencilState = &ds;
749     gp.pColorBlendState = &cb;
750     gp.pDynamicState = &dynInfo;
751     gp.layout = pipelineLayout;
752     gp.renderPass = VK_NULL_HANDLE;
753     gp.subpass = 0;
754
755     if (vkCreateGraphicsPipelines(device, VK_NULL_HANDLE, 1, &gp, nullptr, &skyboxPipeline) != VK_SUCCESS)
756         throw std::runtime_error("Failed to create skybox pipeline");
757
758     vkDestroyShaderModule(device, frag, nullptr);
759     vkDestroyShaderModule(device, vert, nullptr);
760 }
761

```



Exercise 3: IMPLEMENTING REFLECTIONS

Solution: I modified the fragment shader (reflection.frag) for the main object. I added a uniform samplerCube skySampler to the descriptor layout binding.

In the main() function of the shader, I calculated the view vector V (from camera to fragment) and the normal vector N. I used the GLSL built-in function reflect(-V, N) to calculate the reflection vector R.

This vector R was then used to sample the skybox texture: texture(skySampler, R). The result is that the cube perfectly mirrors the surrounding skybox environment.

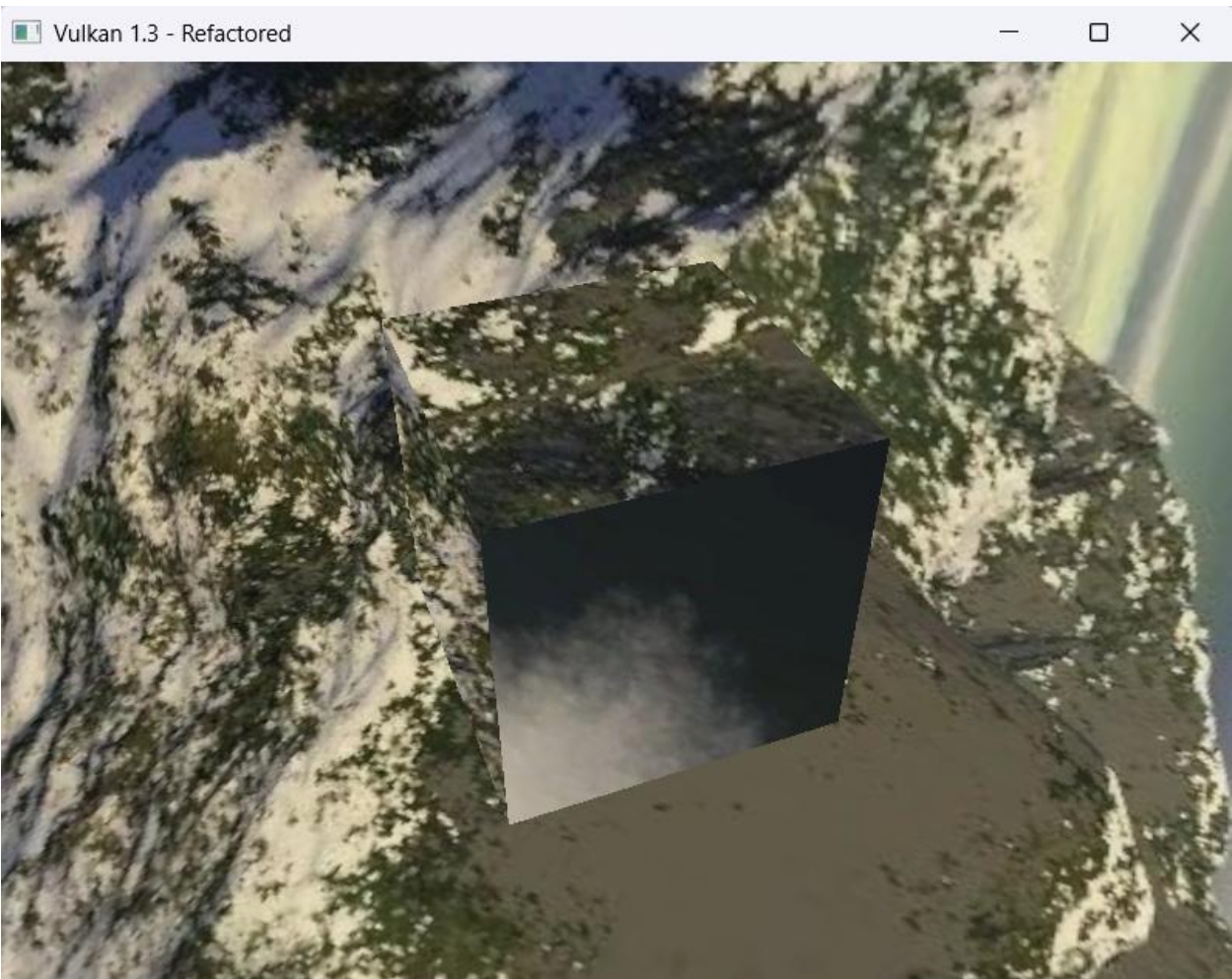
Reflection: This exercise connected the skybox from the previous step to the objects in the scene. It was satisfying to see how simple the math is in the shader—just one line (reflect) creates a complex visual effect. It clarifies that "reflection" in this context isn't ray-tracing geometry, but simply looking up a color from the environment map based on the surface angle.

Question: The reflection I implemented is perfectly sharp, like a mirror. If I wanted to simulate a rougher metal (glossy reflection), would I need to manually blur the reflection vector, or is it better to use the mipmap levels of the cubemap texture to achieve the blurred look efficiently?

```
1  #version 450
2  layout(set=0, binding=0) uniform SceneUBO {
3      mat4 view;
4      mat4 proj;
5      vec3 lightPos1;
6      vec3 lightPos2;
7      vec3 eyePos;
8      vec2 texSize;
9      int filterMode;
10 } ubo;
11
12 layout(set=0, binding=4) uniform samplerCube skySampler;
13
14 layout(location=0) in vec3 inWorldPos;
15 layout(location=1) in vec3 inWorldNormal;
16
17 layout(location=0) out vec4 outColor;
18
19 void main() {
20     vec3 N = normalize(inWorldNormal);
21     vec3 V = normalize(ubo.eyePos - inWorldPos);
22     vec3 R = reflect(-V, N);
23     vec3 reflectionColor = texture(skySampler, R).rgb;
24     outColor = vec4(reflectionColor, 1.0);
25 }
```

reflection.frag ➦ ✕ skybox.vert skybox.frag **shader.vert** ➦ ✕ shader.frag

```
1      #version 450
2
3      layout(set=0,binding=0) uniform SceneUBO {
4          mat4 view;
5          mat4 proj;
6          vec3 lightPos1;
7          vec3 lightPos2;
8          vec3 eyePos;
9          vec2 texSize;
10         int  filterMode;
11     } ubo;
12
13     layout(push_constant) uniform ObjectData {
14         mat4 model;
15         vec4 ambientMat;
16         vec4 specularMat;
17         float shininess;
18         int  texIndex;
19     } pc;
20
21     layout(location=0) in vec3 inPosition;
22     layout(location=1) in vec3 inColor;
23     layout(location=2) in vec3 inNormal;
24
25     layout(location=0) out vec3 fragWorldPos;
26     layout(location=1) out vec3 fragWorldNormal;
27     layout(location=2) out vec2 fragTexCoord;
28
29     void main() {
30         mat4 M = pc.model;
31         vec4 worldPos = M * vec4(inPosition, 1.0);
32         fragWorldPos = worldPos.xyz;
33         fragWorldNormal = normalize(mat3(M) * inNormal);
34
35         // existing UV / other varying assignments...
36         gl_Position = ubo.proj * ubo.view * worldPos;
37     }
```

Exercise 4: IMPLEMENTING REFRACTION

Solution:  As shown in the shader code screenshots, I replaced the reflect function with refract.

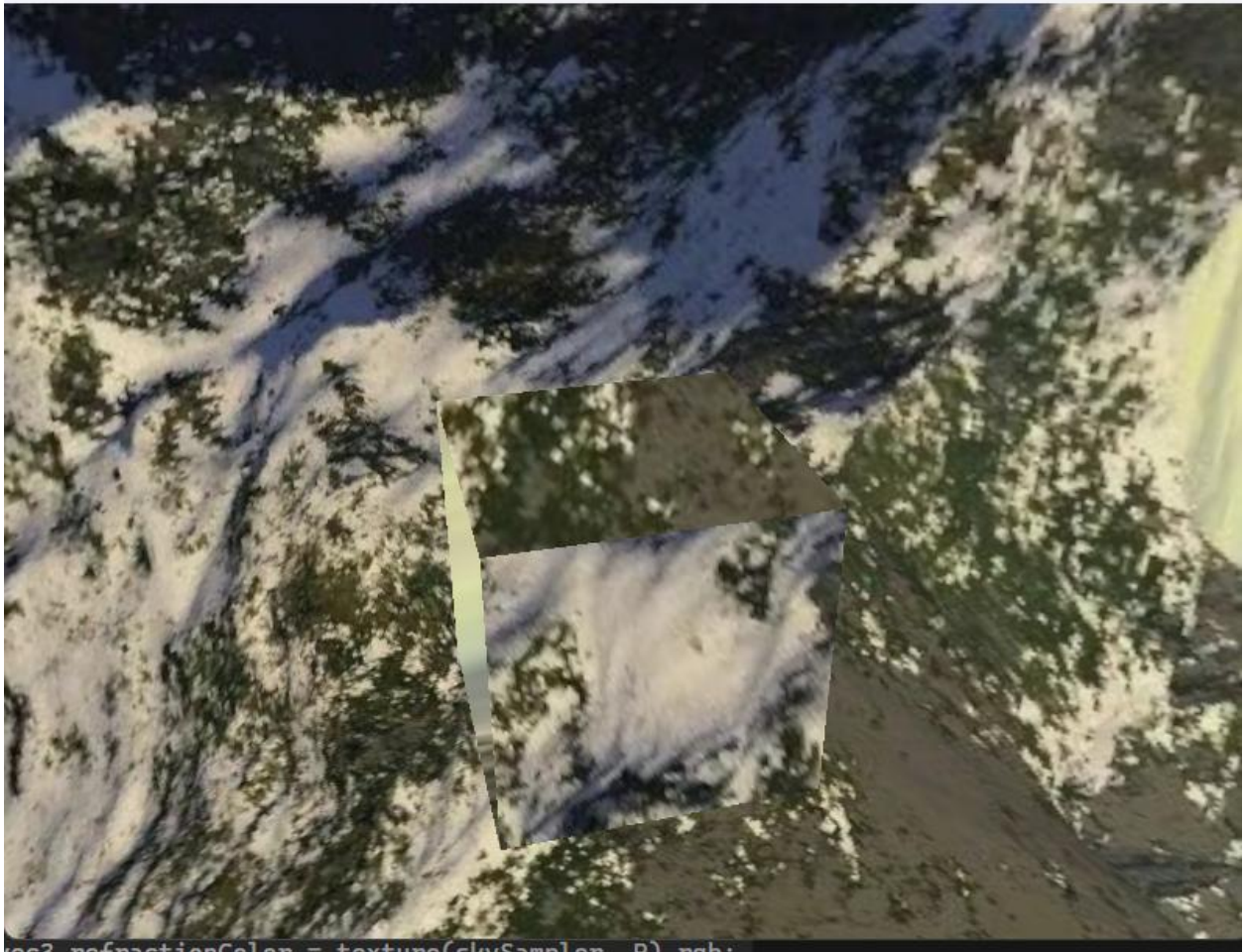
I defined an Index of Refraction (IOR) ratio. The lab suggested 1.00 / 1.33 (Air to Water). I calculated the refraction vector using `vec3 R = refract(-V, N, IOR)`.

I then sampled the skybox using this new vector R. The visual output (shown in the screenshot) distorts the background skybox as it passes through the cube, mimicking the way light bends through a dense material .

Reflection: It is interesting how similar refraction is to reflection in terms of implementation; changing a single function and adding a physical constant (IOR) completely changes the material perception from metal to glass. This highlights the power of PBR (Physically Based Rendering) workflows where material properties drive the shader logic.

Question: Real glass exhibits chromatic aberration (dispersion), where different colors bend at different angles. Would calculating three separate refraction vectors (one for Red, Green, and Blue) with slightly different IOR values be the standard way to implement this effect in Vulkan, or is that too expensive for a real-time shader?


```
shader.vert  X  shader.frag  X  Lab_Tutorial_Template.cpp
2      layout(set=0, binding=0) uniform SceneUBO {
3          mat4 view;
4          mat4 proj;
5          vec3 lightPos1;
6          vec3 lightPos2;
7          vec3 eyePos;
8          vec2 texSize;
9          int  filterMode;
10     } ubo;
11
12     layout(set=0, binding=4) uniform samplerCube skySampler;
13
14     layout(location=0) in vec3 inWorldPos;
15     layout(location=1) in vec3 inWorldNormal;
16
17     layout(location=0) out vec4 outColor;
18
19     void main() {
20         vec3 N = normalize(inWorldNormal);
21         vec3 V = normalize(ubo.eyePos - inWorldPos);
22         float IOR = 1.00 / 1.33; // Index of Refraction (air to water)
23         // Calculate the refraction vector
24         vec3 R = refract(-V, N, IOR);
25         vec3 refractionColor = texture(skySampler, R).rgb;
26         outColor = vec4(refractionColor, 1.0);
27     }
```



```
vec3 refractionColor = texture(skySampler, R).rgb;
```

Exercise 5: SPRITE-BASED PARTICLE SYSTEM

Solution:

- **Geometry:** I created a ParticleVertex struct and a BuildParticleGeometry function that generates a list of quads. The center position is stored, and the corners are defined by offsets (e.g., -1, -1), allowing the vertex shader to expand them.
- **Pipeline:** I created createParticlePipeline with specific blend states. I enabled blending and set srcColorBlendFactor = VK_BLEND_FACTOR_SRC_ALPHA and dstColorBlendFactor = VK_BLEND_FACTOR_ONE. This "Additive Blending" makes overlapping particles brighten the pixels, creating a glowing fire look.
- **Depth:** I set depthWriteEnable = VK_FALSE. This is crucial for transparent particles so they don't occlude each other while still being tested against the solid scene geometry.
- **Billboarding:** In the shader (implied by the output), the quad vertices are aligned to the camera view so the particles always face the viewer.

Reflection: This exercise was the most complex in terms of pipeline management. Understanding why depthWrite must be disabled for transparency was a key learning point—if enabled, the invisible parts of the quad would write to the Z-buffer and block particles behind them. The additive blending successfully created a convincing "hot" fire effect where the center is brightest.

Question: Currently, my particles are generated on the CPU in BuildParticleGeometry and uploaded. For a system with thousands of particles, the CPU-to-GPU bandwidth would be a bottleneck. Would using a **Compute Shader** to update particle positions and a Storage Buffer to render them be the next logical step to optimize this system?

```

126
127 struct ParticleVertex {
128     glm::vec3 center; // world-space center; z also acts as seed in shaders
129     glm::vec2 corner; // (-1,-1), (1,-1), (1,1), (-1,1)
130 };
131
132 struct ParticleUBO {
133     glm::mat4 view;
134     glm::mat4 proj;
135     float time;
136     float _pad0{ 0 }, _pad1{ 0 }, _pad2{ 0 }; // std140 padding
137 };
138

```

```

489
490 // --- Particle system resources ---
491 VkBuffer particleVertexBuffer = VK_NULL_HANDLE;
492 VkDeviceMemory particleVertexMemory = VK_NULL_HANDLE;
493 VkBuffer particleIndexBuffer = VK_NULL_HANDLE;
494 VkDeviceMemory particleIndexMemory = VK_NULL_HANDLE;
495 uint32_t particleIndexCount = 0;
496
497 VkBuffer particleUBOBuffer[MAX_FRAMES_IN_FLIGHT];
498 VkDeviceMemory particleUBOMemory[MAX_FRAMES_IN_FLIGHT];
499 void* particleUBOMapped[MAX_FRAMES_IN_FLIGHT];
500
501 VkDescriptorSetLayout particleDescriptorSetLayout = VK_NULL_HANDLE;
502 VkPipelineLayout particlePipelineLayout = VK_NULL_HANDLE;
503 VkPipeline particlePipeline = VK_NULL_HANDLE;
504 VkDescriptorSet particleDescriptorSets[MAX_FRAMES_IN_FLIGHT];
505
506 void createParticleBuffers(uint32_t particleCount);
507 void createParticleUBOs();
508 void createParticleDescriptorSetLayout();
509 void allocateParticleDescriptorSets();
510 void createParticlePipeline();
511
512
513 void BuildParticleGeometry(std::vector<ParticleVertex>& vertices, std::vector<uint32_t>& indices, uint32_t particleCount);
514 VkVertexInputBindingDescription ParticleBinding();
515 std::array<VkVertexInputAttributeDescription, 2> ParticleAttributes();
516 VkDescriptorSetLayout CreateParticleSetLayout(VkDevice device);
517 void FillParticlePipelineStates(
518     VkPipelineRasterizationStateCreateInfo& rs,
519     VkPipelineDepthStencilStateCreateInfo& ds,
520     VkPipelineColorBlendAttachmentState& cb);
521 void UpdateParticleUBO(ParticleUBO& ubo, const glm::mat4& view, const glm::mat4& proj, float timeSeconds);
522 void DrawParticles(VkCommandBuffer cmd,
523     VkPipeline particlePipeline,
524     VkPipelineLayout particleLayout,
525     VkDescriptorSet particleSet,
526     VkBuffer vbo, VkDeviceSize vboOffset,
527     VkBuffer ibo, VkDeviceSize iboOffset,
528     uint32_t indexCount);
529

```

```

1606
1607 ParticleUBO pubo{};
1608 pubo.view = glm::lookAt(cameraPos, cameraPos + cameraFront, cameraUp);
1609 pubo.proj = glm::perspective(glm::radians(45.0f),
1610     swapChainExtent.width / (float)swapChainExtent.height,
1611     0.1f, 10.0f);
1612 pubo.proj[1][1] *= -1;
1613 pubo.time = (float)glfwGetTime();
1614 pubo.particleSize = .3f;
1615 memcpy(particleUBOMapped[currentFrame], &pubo, sizeof(pubo));
1616
1617 vkResetFences(device, 1, &inFlightFences[currentFrame]);
1618 vkResetCommandBuffer(commandBuffers[currentFrame], 0);
1619 recordCommandBuffer(commandBuffers[currentFrame], imageIndex);
1620
1621 VkCommandBufferSubmitInfo commandBufferInfo{};
1622 commandBufferInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_SUBMIT_INFO;
1623 commandBufferInfo.commandBuffer = commandBuffers[currentFrame];

```

```

1760 // --- Particle pass ---
1761 vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, particlePipeline);
1762 vkCmdBindDescriptorSets(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS,
1763     particlePipelineLayout, 0, 1, &particleDescriptorSets[currentFrame], 0, nullptr);
1764 vkCmdSetViewport(commandBuffer, 0, 1, &vp);
1765 vkCmdSetScissor(commandBuffer, 0, 1, &sc);
1766
1767 VkDeviceSize pOff = 0;
1768 vkCmdBindVertexBuffers(commandBuffer, 0, 1, &particleVertexBuffer, &pOff);
1769 vkCmdBindIndexBuffer(commandBuffer, particleIndexBuffer, 0, VK_INDEX_TYPE_UINT32);
1770 // No push constants needed
1771 vkCmdDrawIndexed(commandBuffer, particleIndexCount, 1, 0, 0, 0);
1772
1773 // --- Main objects pass ---
1774 vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, graphicsPipeline);
1775 vkCmdSetDepthTestEnable(commandBuffer, depthTestEnabled ? VK_TRUE : VK_FALSE);
1776 vkCmdSetDepthWriteEnable(commandBuffer, depthWriteEnabled ? VK_TRUE : VK_FALSE);
1777

```

```

2744
2745 void HelloTriangleApplication::createParticleBuffers(uint32_t particleCount)
2746 {
2747     std::vector<ParticleVertex> pVerts;
2748     std::vector<uint32_t> pIdx;
2749     BuildParticleGeometry(pVerts, pIdx, particleCount);
2750     particleIndexCount = static_cast<uint32_t>(pIdx.size());
2751
2752     VkDeviceSize vSize = sizeof(ParticleVertex) * pVerts.size();
2753     VkDeviceSize iSize = sizeof(uint32_t) * pIdx.size();
2754
2755     VkBuffer stagingV; VkDeviceMemory stagingVM;
2756     createBuffer(vSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
2757         VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
2758         stagingV, stagingVM);
2759     void* data;
2760     vkMapMemory(device, stagingVM, 0, vSize, 0, &data);
2761     memcpy(data, pVerts.data(), (size_t)vSize);
2762     vkUnmapMemory(device, stagingVM);
2763
2764     createBuffer(vSize,
2765         VK_BUFFER_USAGE_TRANSFER_DST_BIT | VK_BUFFER_USAGE_VERTEX_BUFFER_BIT,
2766         VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT,
2767         particleVertexBuffer, particleVertexMemory);
2768     copyBuffer(stagingV, particleVertexBuffer, vSize);
2769     vkDestroyBuffer(device, stagingV, nullptr);
2770     vkFreeMemory(device, stagingVM, nullptr);
2771
2772     VkBuffer stagingI; VkDeviceMemory stagingIM;
2773     createBuffer(iSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
2774         VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
2775         stagingI, stagingIM);
2776     vkMapMemory(device, stagingIM, 0, iSize, 0, &data);
2777     memcpy(data, pIdx.data(), (size_t)iSize);
2778     vkUnmapMemory(device, stagingIM);
2779
2780     createBuffer(iSize,
2781         VK_BUFFER_USAGE_TRANSFER_DST_BIT | VK_BUFFER_USAGE_INDEX_BUFFER_BIT,
2782         VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT,
2783         particleIndexBuffer, particleIndexMemory);
2784     copyBuffer(stagingI, particleIndexBuffer, iSize);
2785     vkDestroyBuffer(device, stagingI, nullptr);
2786     vkFreeMemory(device, stagingIM, nullptr);
2787 }

```



```

2788
2789 void HelloTriangleApplication::createParticleUBOs()
2790 {
2791     for (int i = 0; i < MAX_FRAMES_IN_FLIGHT; ++i) {
2792         VkDeviceSize sz = sizeof(ParticleUBO);
2793         createBuffer(sz, VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT,
2794             VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
2795             particleUBOBuffer[i], particleUBOMemory[i]);
2796         vkMapMemory(device, particleUBOMemory[i], 0, sz, 0, &particleUBOMapped[i]);
2797     }
2798 }
2799
2800 void HelloTriangleApplication::createParticleDescriptorSetLayout()
2801 {
2802     particleDescriptorSetLayout = CreateParticleSetLayout(device);
2803 }
2804
2805 void HelloTriangleApplication::allocateParticleDescriptorSets()
2806 {
2807     VkDescriptorSetLayout layouts[MAX_FRAMES_IN_FLIGHT];
2808     for (int i = 0; i < MAX_FRAMES_IN_FLIGHT; ++i) layouts[i] = particleDescriptorSetLayout;
2809
2810     VkDescriptorSetAllocateInfo alloc{ VK_STRUCTURE_TYPE_DESCRIPTOR_SET_ALLOCATE_INFO };
2811     alloc.descriptorPool = descriptorPool; // reuse existing pool (enough space)
2812     alloc.descriptorSetCount = MAX_FRAMES_IN_FLIGHT;
2813     alloc.pSetLayouts = layouts;
2814     if (vkAllocateDescriptorSets(device, &alloc, particleDescriptorSets) != VK_SUCCESS)
2815         throw std::runtime_error("Failed to allocate particle descriptor sets");
2816
2817     for (int i = 0; i < MAX_FRAMES_IN_FLIGHT; ++i) {
2818         VkDescriptorBufferInfo buf{};
2819         buf.buffer = particleUBOBuffer[i];
2820         buf.offset = 0;
2821         buf.range = sizeof(ParticleUBO);
2822
2823         VkWriteDescriptorSet w{};
2824         w.sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
2825         w.dstSet = particleDescriptorSets[i];
2826         w.dstBinding = 0;
2827         w.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
2828         w.descriptorCount = 1;
2829         w.pBufferInfo = &buf;
2830         vkUpdateDescriptorSets(device, 1, &w, 0, nullptr);
2831     }
2832 }

```

```

2834 void HelloTriangleApplication::createParticlePipeline() {
2835     auto vertCode = readFile("shaders/particle.vert.spv");
2836     auto fragCode = readFile("shaders/particle.frag.spv");
2837     VkShaderModule vmod = createShaderModule(vertCode);
2838     VkShaderModule fmod = createShaderModule(fragCode);
2839
2840     VkPipelineShaderStageCreateInfo stages[2]{};
2841     stages[0].sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
2842     stages[0].stage = VK_SHADER_STAGE_VERTEX_BIT;
2843     stages[0].module = vmod;
2844     stages[0].pName = "main";
2845     stages[1].sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
2846     stages[1].stage = VK_SHADER_STAGE_FRAGMENT_BIT;
2847     stages[1].module = fmod;
2848     stages[1].pName = "main";
2849
2850     auto binding = ParticleBinding();
2851     auto attrs = ParticleAttributes();
2852     VkPipelineVertexInputStateCreateInfo vi{ VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO };
2853     vi.vertexBindingDescriptionCount = 1;
2854     vi.pVertexBindingDescriptions = &binding;
2855     vi.vertexAttributeDescriptionCount = (uint32_t)attrs.size();
2856     vi.pVertexAttributeDescriptions = attrs.data();
2857
2858     VkPipelineInputAssemblyStateCreateInfo ia{ VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_INFO };
2859     ia.topology = VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST;
2860
2861     VkPipelineViewportStateCreateInfo vp{ VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO };
2862     vp.viewportCount = 1;
2863     vp.scissorCount = 1;
2864
2865     VkPipelineRasterizationStateCreateInfo rs;
2866     VkPipelineDepthStencilStateCreateInfo ds;
2867     VkPipelineColorBlendAttachmentState cbAtt;
2868     FillParticlePipelineStates(rs, ds, cbAtt);
2869
2870     VkPipelineMultisampleStateCreateInfo ms{ VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO };
2871     ms.rasterizationSamples = VK_SAMPLE_COUNT_1_BIT;
2872
2873     VkPipelineColorBlendStateCreateInfo cb{ VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_STATE_CREATE_INFO };
2874     cb.attachmentCount = 1;
2875     cb.pAttachments = &cbAtt;
2876
2877     std::vector<VkDynamicState> dyn = {
2878         VK_DYNAMIC_STATE_VIEWPORT

```

```

2920 void HelloTriangleApplication::BuildParticleGeometry(std::vector<ParticleVertex>& vertices, std::vector<uint32_t>& indices, uint32_t particleCount)
2921 {
2922     vertices.clear();
2923     indices.clear();
2924     const glm::vec2 corners[4] = { {-1,-1}, { 1,-1}, { 1, 1}, {-1, 1} };
2925
2926     for (uint32_t i = 0; i < particleCount; ++i) {
2927         float seed = (particleCount > 1) ? (float)i / float(particleCount - 1) : 0.0f;
2928         glm::vec3 center = glm::vec3(0.0f, 0.0f, seed); // xy at origin; z used as a unique seed
2929
2930         uint32_t base = static_cast<uint32_t>(vertices.size());
2931         for (int c = 0; c < 4; ++c) {
2932             vertices.push_back(ParticleVertex{ center, corners[c] });
2933         }
2934         // two triangles per quad
2935         indices.insert(indices.end(), { base + 0, base + 1, base + 2, base + 2, base + 3, base + 0 });
2936     }
2937 }
2938
2939 VkVertexInputBindingDescription HelloTriangleApplication::ParticleBinding()
2940 {
2941     VkVertexInputBindingDescription b{};
2942     b.binding = 0;
2943     b.stride = sizeof(ParticleVertex);
2944     b.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;
2945     return b;
2946 }
2947
2948 std::array<VkVertexInputAttributeDescription, 2> HelloTriangleApplication::ParticleAttributes()
2949 {
2950     std::array<VkVertexInputAttributeDescription, 2> a{};
2951     a[0] = { 0, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(ParticleVertex, center) }; // inParticlePos
2952     a[1] = { 1, 0, VK_FORMAT_R32G32_SFLOAT,   offsetof(ParticleVertex, corner) }; // inCornerOffset
2953     return a;
2954 }
2955
2956 VkDescriptorSetLayout HelloTriangleApplication::CreateParticleSetLayout(VkDevice device)
2957 {
2958     VkDescriptorSetLayoutBinding uboBinding{};
2959     uboBinding.binding = 0;
2960     uboBinding.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
2961     uboBinding.descriptorCount = 1;
2962     uboBinding.stageFlags = VK_SHADER_STAGE_VERTEX_BIT; // only vertex needs it
2963     uboBinding.pImmutableSamplers = nullptr;
2964
2965     VkDescriptorSetLayoutCreateInfo info{ VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO };
2966     info.bindingCount = 1;
2967     info.pBindings = &uboBinding;

```

```

888
889 void HelloTriangleApplication::cleanup() {
890     cleanupSwapChain();
891
892     vkDestroyPipeline(device, graphicsPipeline, nullptr);
893     vkDestroyPipelineLayout(device, pipelineLayout, nullptr);
894     vkDestroyDescriptorSetLayout(device, descriptorSetLayout, nullptr);
895
896     vkDestroyBuffer(device, indexBuffer, nullptr);
897     vkFreeMemory(device, indexBufferMemory, nullptr);
898
899     vkDestroyBuffer(device, vertexBuffer, nullptr);
900     vkFreeMemory(device, vertexBufferMemory, nullptr);
901
902     for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
903         vkDestroyBuffer(device, uniformBuffers[i], nullptr);
904         vkFreeMemory(device, uniformBuffersMemory[i], nullptr);
905     }
906     vkDestroyDescriptorPool(device, descriptorPool, nullptr);
907
908     for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
909         vkDestroySemaphore(device, renderFinishedSemaphores[i], nullptr);
910         vkDestroySemaphore(device, imageAvailableSemaphores[i], nullptr);
911         vkDestroyFence(device, inFlightFences[i], nullptr);
912
913         vkDestroyBuffer(device, particleUBOBuffer[i], nullptr);
914         vkFreeMemory(device, particleUBOMemory[i], nullptr);
915     }
916     vkDestroyBuffer(device, particleIndexBuffer, nullptr);
917     vkFreeMemory(device, particleIndexMemory, nullptr);
918     vkDestroyBuffer(device, particleVertexBuffer, nullptr);
919     vkFreeMemory(device, particleVertexMemory, nullptr);
920     vkDestroyPipeline(device, particlePipeline, nullptr);
921     vkDestroyPipelineLayout(device, particlePipelineLayout, nullptr);
922     vkDestroyDescriptorSetLayout(device, particleDescriptorSetLayout, nullptr);
923
924     vkDestroyCommandPool(device, commandPool, nullptr);
925     vkDestroyDevice(device, nullptr);
926
927     if (enableValidationLayers) {
928         DestroyDebugUtilsMessengerEXT(instance, debugMessenger, nullptr);
929     }
930
931     vkDestroySurfaceKHR(instance, surface, nullptr);
932     vkDestroyInstance(instance, nullptr);
933

```

```

2974 void HelloTriangleApplication::FillParticlePipelineStates(VkPipelineRasterizationStateCreateInfo& rs, VkPipelineDepthStencilStateCreateInfo& ds, VkPipelineColorBlendAttache
2975 {
2976     rs = { VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO };
2977     rs.polygonMode = VK_POLYGON_MODE_FILL;
2978     rs.cullMode = VK_CULL_MODE_NONE;
2979     rs.frontFace = VK_FRONT_FACE_COUNTER_CLOCKWISE;
2980     rs.lineWidth = 1.0f;
2981
2982     ds = { VK_STRUCTURE_TYPE_PIPELINE_DEPTH_STENCIL_STATE_CREATE_INFO };
2983     ds.depthTestEnable = VK_TRUE;
2984     ds.depthWriteEnable = VK_FALSE; // important for blending among particles
2985     ds.depthCompareOp = VK_COMPARE_OP_LESS_OR_EQUAL;
2986
2987     cb = {};
2988     cb.blendEnable = VK_TRUE;
2989     cb.srcColorBlendFactor = VK_BLEND_FACTOR_SRC_ALPHA;
2990     cb.dstColorBlendFactor = VK_BLEND_FACTOR_ONE; // additive fire
2991     cb.colorBlendOp = VK_BLEND_OP_ADD;
2992     cb.srcAlphaBlendFactor = VK_BLEND_FACTOR_ONE;
2993     cb.dstAlphaBlendFactor = VK_BLEND_FACTOR_ONE_MINUS_SRC_ALPHA;
2994     cb.alphaBlendOp = VK_BLEND_OP_ADD;
2995     cb.colorWriteMask = VK_COLOR_COMPONENT_R_BIT | VK_COLOR_COMPONENT_G_BIT |
2996         VK_COLOR_COMPONENT_B_BIT | VK_COLOR_COMPONENT_A_BIT;
2997 }
2998
2999 void HelloTriangleApplication::UpdateParticleUBO(ParticleUBO& ubo, const glm::mat4& view, const glm::mat4& proj, float timeSeconds)
3000 {
3001     ubo.view = view;
3002     ubo.proj = proj;
3003     ubo.time = timeSeconds;
3004 }
3005
3006 void HelloTriangleApplication::DrawParticles(VkCommandBuffer cmd, VkPipeline particlePipeline, VkPipelineLayout particleLayout, VkDescriptorSet particleSet, VkBuffer vbo, Vk
3007 {
3008     vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, particlePipeline);
3009     vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, particleLayout, 0, 1, &particleSet, 0, nullptr);
3010
3011     VkBuffer buffers[] = { vbo };
3012     VkDeviceSize offsets[] = { vboOffset };
3013     vkCmdBindVertexBuffers(cmd, 0, 1, buffers, offsets);
3014     vkCmdBindIndexBuffer(cmd, ibo, iboOffset, VK_INDEX_TYPE_UINT32);
3015
3016     vkCmdDrawIndexed(cmd, indexCount, 1, 0, 0, 0);
3017 }

```

